Code Execution - Machine Learning Module UoEO



Group: Lorem Ipsum · September 5th, 2026

Contenido

Business question

Notebook execution guidelines

Data Preparation

Data analysis

Data cleaning

Enhance dataset

Clustering Analysis

Real Estate Volatility by NTA Explaination

Business question

> *How can we develop an Automated Valuation Model (AVM) to provide a cold-start asking price for new Airbnb listings based on their features, and which geographic segments present the highest pricing volatility?*

Notebook execution guidelines

A kaggle username and its PAT is needed to retrieve the data, get yours from https://www.kaggle.com/settings/api then generate a "Legacy API Credentials". This data goes within the `.env` file.

Before running this notebook, initialize a virtual environment and install requirements:

1. `python -m venv .venv`: to create the virtual environment.
2. `.venv\scripts\activate`: to activate the virtual environment using Windows.
3. `pip install -r requirements.txt`: to install the list of requirements.

In [33]:

```
import kaggle
import warnings
import numpy as np
import pandas as pd
from pathlib import Path
from dotenv import load_dotenv
import geopandas as gpd
import matplotlib.pyplot as plt
from sklearn.neighbors import BallTree
from sklearn.model_selection import (
    train_test_split,
    cross_val_predict,
    KFold,
    GridSearchCV,
)
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import (
    r2_score,
    mean_absolute_error,
    mean_squared_error,
    make_scorer,
)
from sklearn.decomposition import PCA

warnings.filterwarnings("ignore")

RANDOM_SEED = 85
```

```
load_dotenv()
datasets_path = Path("dataset")
```

Download and read file (requires Kaggle account):

In [34]:

```
if not (datasets_path / "AB_NYC_2019.csv").exists():
    kaggle.api.dataset_download_files(
        "dgomonov/new-york-city-airbnb-open-data", path=datasets_path, unzip=True
    )
df_airbnb = pd.read_csv(datasets_path / "AB_NYC_2019.csv")
```

In [35]:

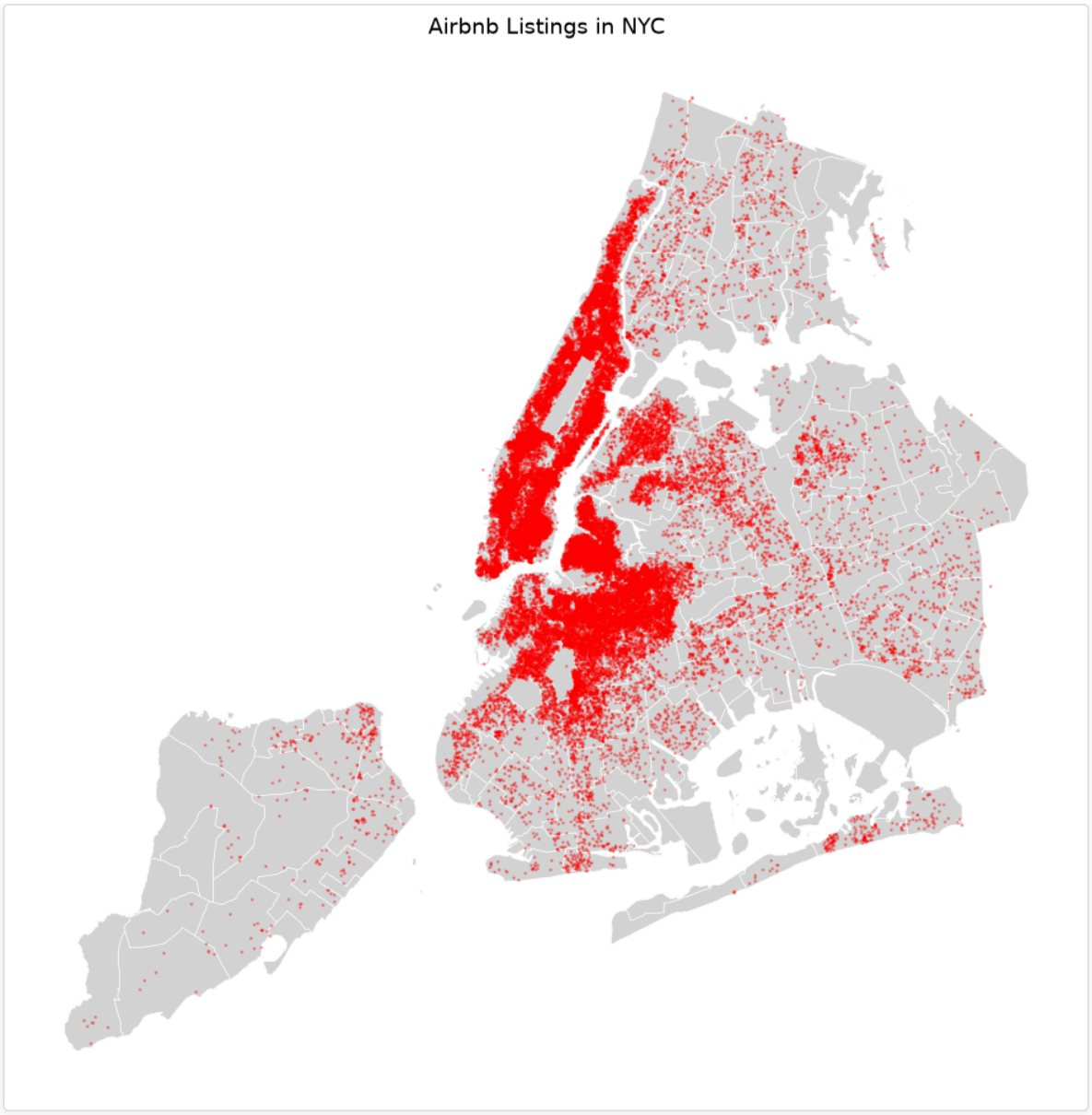
```
# City of New York: 2020 Neighborhood Tabulation Areas (NTAs) - Mapped
# https://data.cityofnewyork.us/City-Government/2020-Neighborhood-Tabulation-Areas-NTAs-Mapped/4hft-v355
url = "https://data.cityofnewyork.us/resource/9nt8-h7nd.geojson"
nyc_geojson = gpd.read_file(url)

# convert airbnb dataframe to geodataframe
gdf_airbnb = gpd.GeoDataFrame(
    df_airbnb,
    geometry=gpd.points_from_xy(df_airbnb.longitude, df_airbnb.latitude),
    crs="EPSG:4326",
)
nyc_geojson = nyc_geojson.to_crs("EPSG:4326")  # ensure same coordinate reference system
airbnb_enriched = gpd.sjoin(
    gdf_airbnb,
    nyc_geojson[["ntaname", "boroname", "geometry"]],
    how="left",
    predicate="within",
)  # assign neighborhoods and boroughs to airbnb listings

fig, ax = plt.subplots(figsize=(12, 12))
nyc_geojson.plot(
    ax=ax, color="lightgrey", edgecolor="white", linewidth=0.5
)  # draw base map of NYC neighborhoods
airbnb_enriched.plot(
    ax=ax, color="red", markersize=1, alpha=0.3
)  # Overlay airbnb listings on top of the NYC map
ax.set_title("Airbnb Listings in NYC", fontsize=14)
ax.set_axis_off()

plt.show()
```

Out [35]:



In [36]:

```
count_neighborhoods = airbnb_enriched["ntaname"].value_counts().reset_index()
count_neighborhoods.columns = ["ntaname", "total_listings"]

density_map = nyc_geojson.merge(
    count_neighborhoods, on="ntaname", how="left"
)  # join the counts with the GeoDataFrame to prepare for choropleth mapping

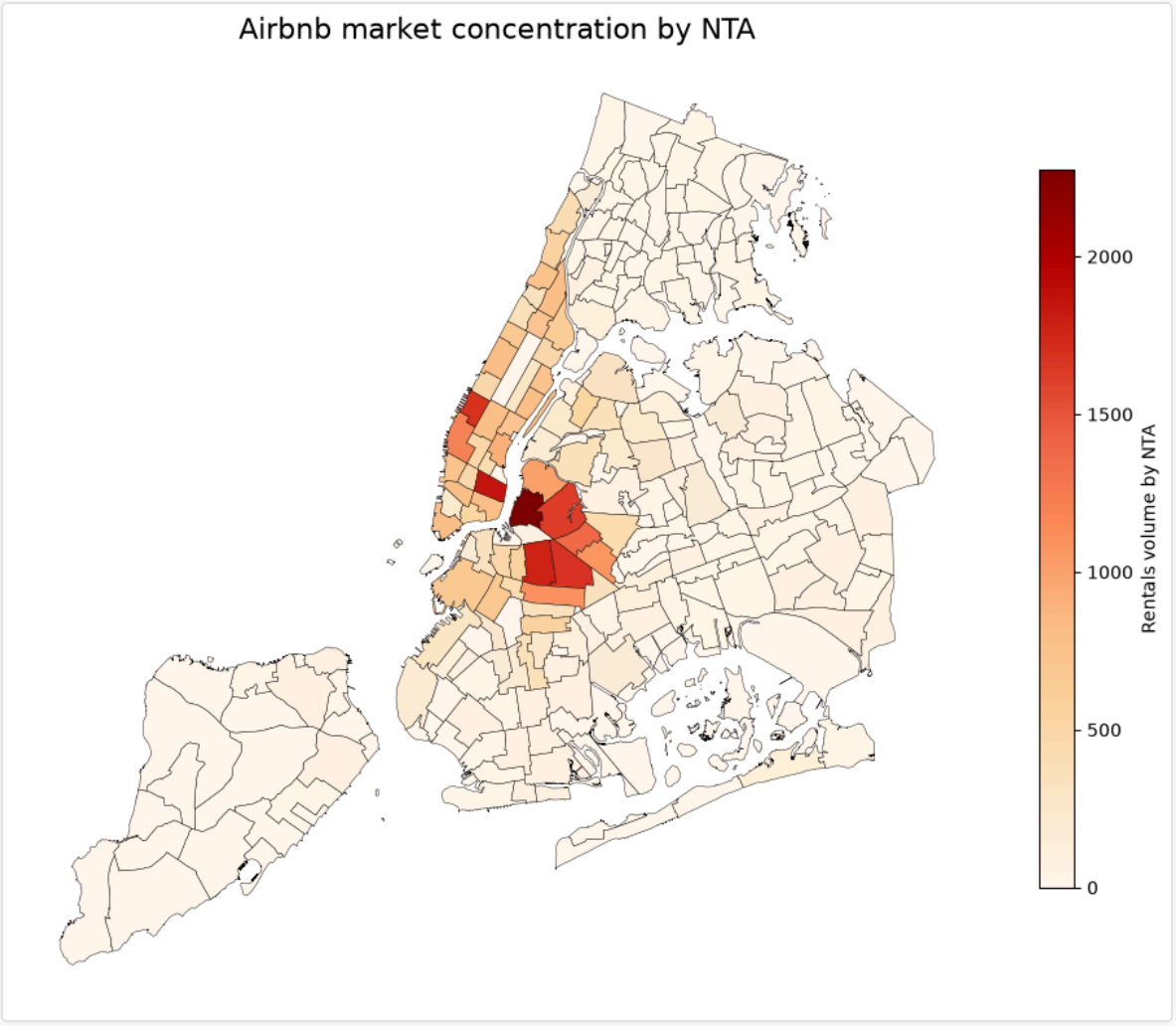
density_map["total_listings"] = density_map["total_listings"].fillna(
    0
)  # fill NaN values with 0 for neighborhoods with no listings
fig, ax = plt.subplots(figsize=(12, 12))

density_map.plot(
    column="total_listings",
    cmap="OrRd",
    linewidth=0.3,
    edgecolor="black",
    legend=True,
    legend_kwds={"shrink": 0.6, "label": "Rentals volume by NTA"},
    ax=ax,
)

ax.set_title("Airbnb market concentration by NTA", fontsize=16)
ax.set_axis_off()
```

```
plt.show()
```

Out [36]:



Data Preparation

Data analysis

Explore how data is composed, then cleanup for ML modeling.

Data cleaning

1. Remove null and missing values
2. Remove outliers
3. One-Hot Encoding for critical variables
4. Standardise geospatial data

In [37]:

```
print("The field name of data: ", df_airbnb.columns)  # The field name of data
print("Number of fields in data: ", len(df_airbnb.columns))  # Number of fields in data
print("Number of data in data: ", len(df_airbnb))  # Number of data in data
```

Out [37]:

```
The field name of data:  Index(['id', 'name', 'host_id', 'host_name', 'neighbourhood_group',
       'neighbourhood', 'latitude', 'longitude', 'room_type', 'price',
       'minimum_nights', 'number_of_reviews', 'last_review',
       'reviews_per_month', 'calculated_host_listings_count',
       'availability_365'],
```

```
      dtype='str')
Number of fields in data:  16
Number of data in data:  48895
```

In [38]:

```
df_clean = airbnb_enriched.copy()
df_clean = df_clean.dropna(
    subset=["ntaname"]
)  # drop rows where 'ntaname' is NaN, which indicates listings outside of NYC boundaries
df_clean["reviews_per_month"] = df_clean["reviews_per_month"].fillna(
    0
)  # fill NaN values in 'reviews_per_month' with 0, indicating no reviews for those listings

# removing outliers
max_price = df_clean["price"].quantile(
    0.99
)  # price 0 is an error, removing the top 1% of prices to avoid skewing the model
df_clean = df_clean[(df_clean["price"] > 0) & (df_clean["price"] <= max_price)]
df_clean = df_clean[
    df_clean["minimum_nights"] <= 365
]  # minimum_nights greater than 365 are likely errors or special cases, so we filter them out

# remove unnecesary columns for ML
ml_columns = [
    "ntaname",
    "boroname",
    "latitude",
    "longitude",
    "room_type",
    "price",
    "minimum_nights",
    "number_of_reviews",
    "reviews_per_month",
    "calculated_host_listings_count",
    "availability_365",
]

df_ml = df_clean[ml_columns].copy()
df_ml["log_price"] = np.log1p(
    df_ml["price"]
)  # log1p is used to avoid issues with log(0) and to handle the skewness of the price distribution

# Export
df_ml.to_csv(datasets_path / "airbnb_nyc_ml_base.csv", index=False)
df_ml.info()
```

Out [38]:

```
<class 'pandas.DataFrame'>
Index: 48383 entries, 0 to 48894
Data columns (total 12 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   ntaname                         48383 non-null  str
 1   boroname                        48383 non-null  str
 2   latitude                        48383 non-null  float64
 3   longitude                       48383 non-null  float64
 4   room_type                       48383 non-null  str
 5   price                           48383 non-null  int64
 6   minimum_nights                  48383 non-null  int64
 7   number_of_reviews               48383 non-null  int64
 8   reviews_per_month               48383 non-null  float64
 9   calculated_host_listings_count  48383 non-null  int64
 10  availability_365                48383 non-null  int64
 11  log_price                       48383 non-null  float64
dtypes: float64(4), int64(5), str(3)
memory usage: 4.8 MB
```

Enhance dataset

Adding distance to public transportation, and city tourist attractions.

In [39]:

```
url_mta = "https://data.ny.gov/api/views/i9wp-a4ja/rows.csv?accessType=DOWNLOAD"
df_mta = pd.read_csv(url_mta)
col_lat_mta = (
    "Entrance Latitude" if "Entrance Latitude" in df_mta.columns else "Latitude"
)
col_lon_mta = (
    "Entrance Longitude" if "Entrance Longitude" in df_mta.columns else "Longitude"
)
df_mta = df_mta.dropna(subset=[col_lat_mta, col_lon_mta])
airbnb_coords = np.radians(
    df_ml[["latitude", "longitude"]].values
)  # convert coordiantes to radians
mta_coords = np.radians(
    df_mta[[col_lat_mta, col_lon_mta]].values
)  # convert coordiantes to radians
tree = BallTree(
    mta_coords, metric="haversine"
)  # create space tree based on subway stations
distances_rad, indices = tree.query(
    airbnb_coords, k=1
)  # Find nearest station for each airbnb listing
df_ml["dist_meters_to_subway"] = (
    distances_rad.flatten() * 6371000
)  # Convert radians to meters

# Check
print(df_ml[["latitude", "longitude", "dist_meters_to_subway"]].head())
```

Out [39]:

```
   latitude  longitude  dist_meters_to_subway
0  40.64749  -73.97237             392.684521
1  40.75362  -73.98377              82.204337
2  40.80902  -73.94190             306.989382
3  40.68514  -73.95976             404.968892
4  40.79851  -73.94399             177.048398
```

In [40]:

```
# Calculate No-ML Baseline (Median by neighborhood and room type)
df_ml["baseline_log_price"] = df_ml.groupby(["boroname", "room_type"])[
    "log_price"
].transform("median")
baseline_r2 = r2_score(df_ml["log_price"], df_ml["baseline_log_price"])
print(f"R2 Score of No-ML Baseline (Median Lookup): {baseline_r2:.4f}")
```

Out [40]:

```
R2 Score of No-ML Baseline (Median Lookup): 0.4902
```

In [41]:

```
# adding points of interest: we considered "https://data.cityofnewyork.us/City-Government/CommonPlace/rxuy-2muj" but has too many
# we are going with strategic points of interest that are relevant to tourists and visitors of NYC
pois = [
    ("Times Square", 40.7580, -73.9855),
    ("Central Park", 40.7644, -73.9730),
    ("Met Museum", 40.7794, -73.9632),
    ("Empire State", 40.7484, -73.9857),
    ("Washington Square", 40.7308, -73.9973),
    ("SoHo", 40.7233, -73.9988),
    ("Financial District", 40.7074, -74.0113),
    ("Penn Station", 40.7505, -73.9934),
    ("Grand Central", 40.7527, -73.9772),
    ("Williamsburg", 40.7160, -73.9587),
    ("Barclays Center", 40.6826, -73.9754),
    ("Long Island City", 40.7465, -73.9455),
]
poi_coords = np.radians([[lat, lon] for _, lat, lon in pois])
airbnb_coords = np.radians(df_ml[["latitude", "longitude"]].values)
tree_poi = BallTree(poi_coords, metric="haversine")
```

```
dist_rad, _ = tree_poi.query(airbnb_coords, k=1)
df_ml["dist_min_poi_meters"] = dist_rad.flatten() * 6371000
```

Clustering Analysis

The purpose of this clustering analysis is to group Airbnb listings with similar characteristics.
This helps identify different market segments, such as budget listings, premium listings, highly available listings, and listings close to tourist attractions or public transport.

In [42]:

```
# Select numerical features that describe listing price, demand, host activity,
# availability, and location accessibility.
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

clustering_features = [
    "price",
    "minimum_nights",
    "number_of_reviews",
    "reviews_per_month",
    "calculated_host_listings_count",
    "availability_365",
    "dist_meters_to_subway",
    "dist_min_poi_meters",
]

df_cluster = df_ml[clustering_features].copy()

# K-Means is distance-based, so all features must be scaled.
# Without scaling, large-value features such as distances or availability would dominate the clustering.
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_cluster_scaled = scaler.fit_transform(df_cluster)
```

In [43]:

```
inertias = []
silhouette_scores = []
k_values = range(2, 12)

# Compare multiple values of k to choose a suitable number of clusters.
for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=RANDOM_SEED, n_init=10)

    labels = kmeans.fit_predict(X_cluster_scaled)

    # Inertia measures within-cluster compactness and is used for the Elbow Method.
    inertias.append(kmeans.inertia_)

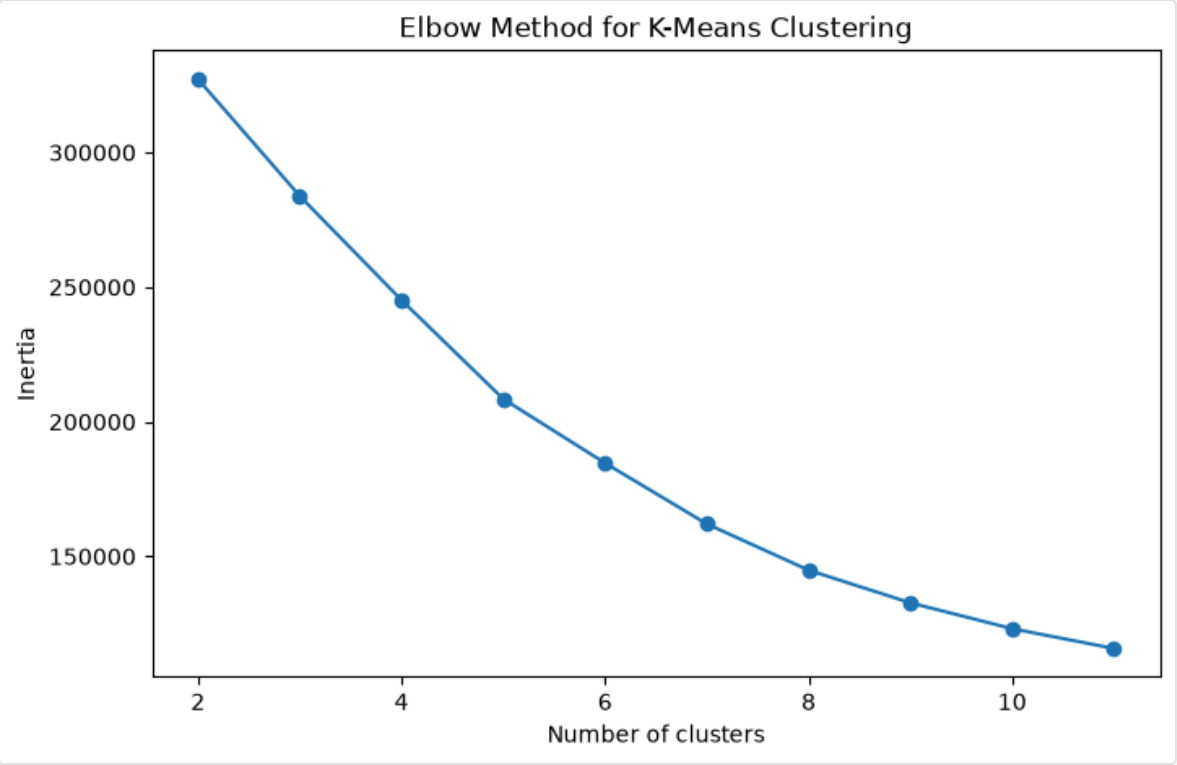
    # Silhouette score measures how well-separated the clusters are.
    score = silhouette_score(X_cluster_scaled, labels, random_state=RANDOM_SEED)

    silhouette_scores.append(score)
```

In [44]:

```
plt.figure(figsize=(8, 5))
plt.plot(k_values, inertias, marker="o")
plt.xlabel("Number of clusters")
plt.ylabel("Inertia")
plt.title("Elbow Method for K-Means Clustering")
plt.show()
```

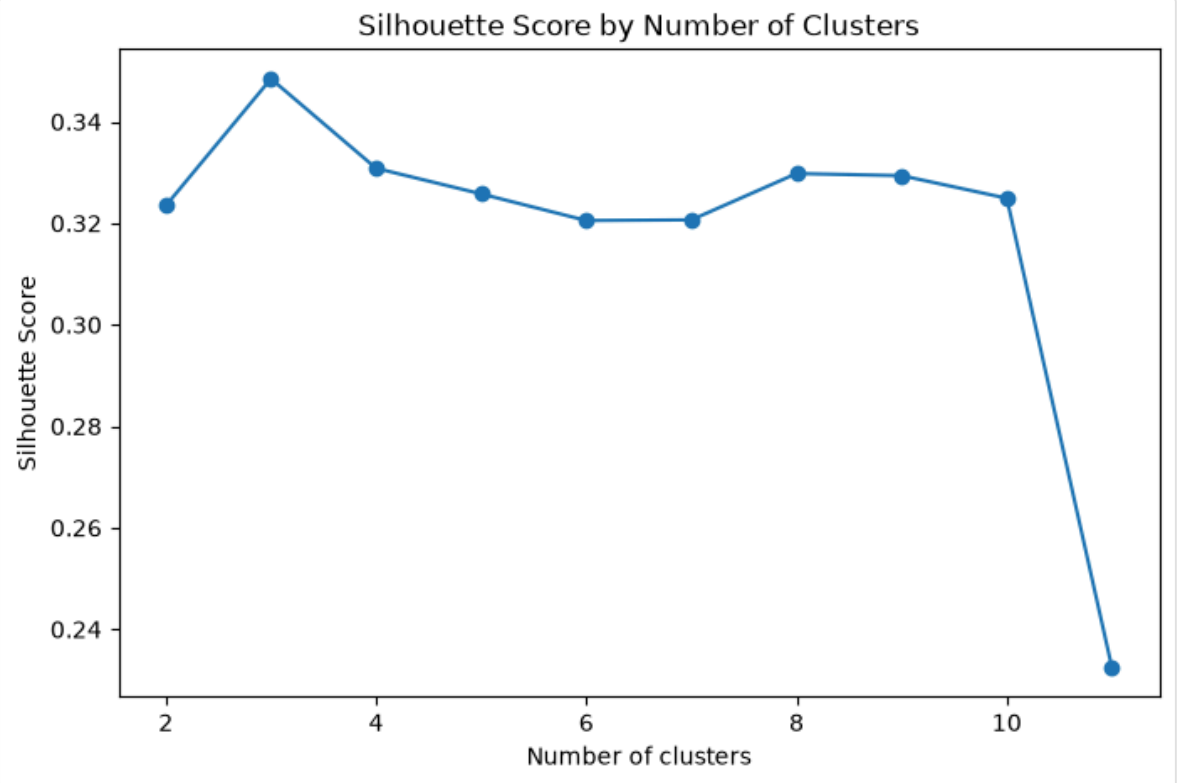
Out [44]:



In [45]:

```
plt.figure(figsize=(8, 5))
plt.plot(k_values, silhouette_scores, marker="o")
plt.xlabel("Number of clusters")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score by Number of Clusters")
plt.show()
```

Out [45]:



In [46]:

```
# Train the final K-Means model using the selected number of clusters.
# k=3 was selected based on the highest Silhouette Score and the Elbow Method.
best_k = 3

kmeans = KMeans(n_clusters=best_k, random_state=RANDOM_SEED, n_init=10)

df_ml["cluster"] = kmeans.fit_predict(X_cluster_scaled)
```

In [47]:

```
# Create a numerical profile for each cluster.
# This helps interpret what each group of listings represents.
cluster_profile = df_ml.groupby("cluster")[clustering_features].mean().round(2)

# Count how many listings belong to each cluster.
cluster_sizes = df_ml["cluster"].value_counts().sort_index()

# Combine average feature values and cluster sizes into one summary table.
cluster_summary = cluster_profile.copy()
cluster_summary["number_of_listings"] = cluster_sizes

cluster_summary
```

Out [47]:

	price	minimum_nights	number_of_reviews	reviews_per_month	calculated_host_listings_count	availability_365	dist_meters_to_subway	dist_min_poi_meters	number_of_li
cluster									
0	141.92	6.48	9.07	0.52	2.75	83.92	300.60	2449.65	36206
1	225.47	47.48	2.87	0.38	164.45	275.07	240.13	1050.20	1354
2	112.02	2.71	74.02	3.13	2.34	185.70	716.39	5781.60	10823

In [48]:

```
# Analyze room type distribution within each cluster.
# This helps determine whether clusters represent different property types.
room_type_distribution = pd.crosstab(
    df_ml["cluster"], df_ml["room_type"], normalize="index"
).round(2)

room_type_distribution
```

Out [48]:

room_type	Entire home/apt	Private room	Shared room
cluster			
0	0.52	0.45	0.02
1	0.88	0.11	0.00
2	0.45	0.52	0.03

In [49]:

```
# Analyze borough distribution within each cluster.
# This connects the clustering results to the geographic business question.
borough_distribution = pd.crosstab(
    df_ml["cluster"], df_ml["boroname"], normalize="index"
).round(2)

borough_distribution
```

Out [49]:

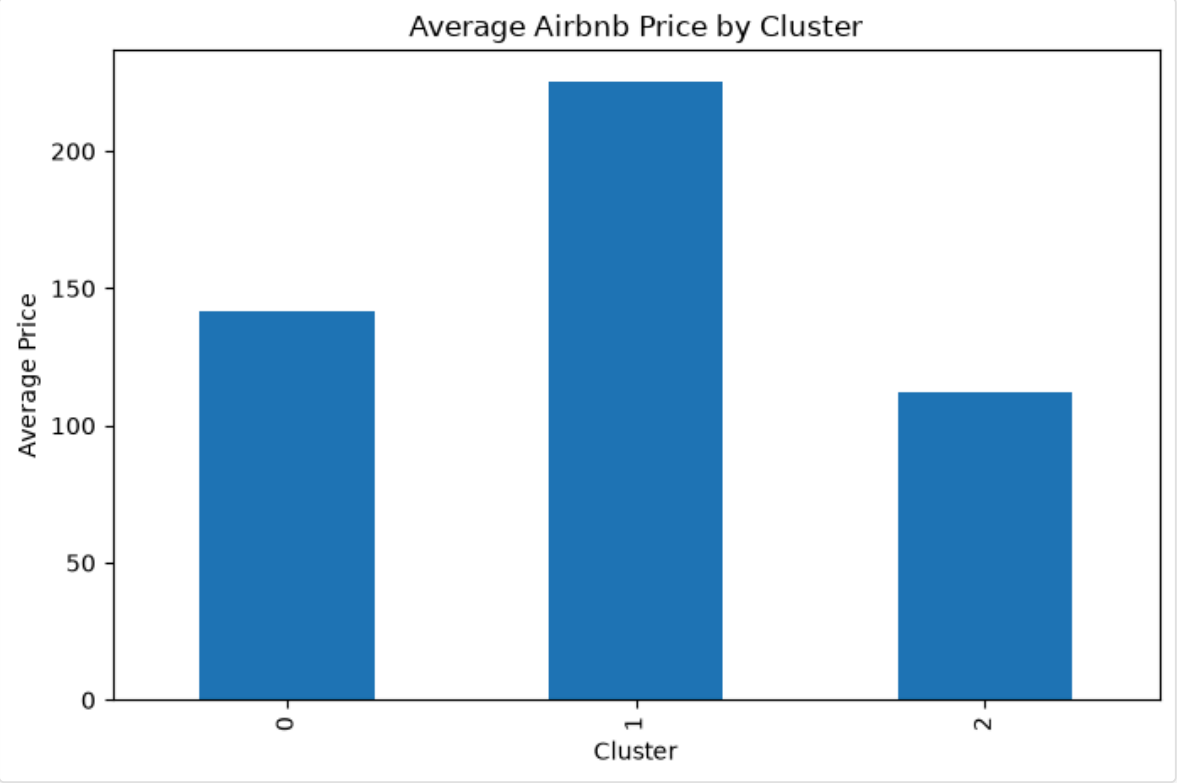
boroname	Bronx	Brooklyn	Manhattan	Queens	Staten Island
cluster					
0	0.01	0.43	0.47	0.08	0.00
1	0.00	0.04	0.90	0.06	0.00
2	0.06	0.39	0.28	0.24	0.03

In [50]:

```
cluster_summary["price"].plot(kind="bar", figsize=(8, 5))

plt.xlabel("Cluster")
plt.ylabel("Average Price")
plt.title("Average Airbnb Price by Cluster")
plt.show()
```

Out [50]:

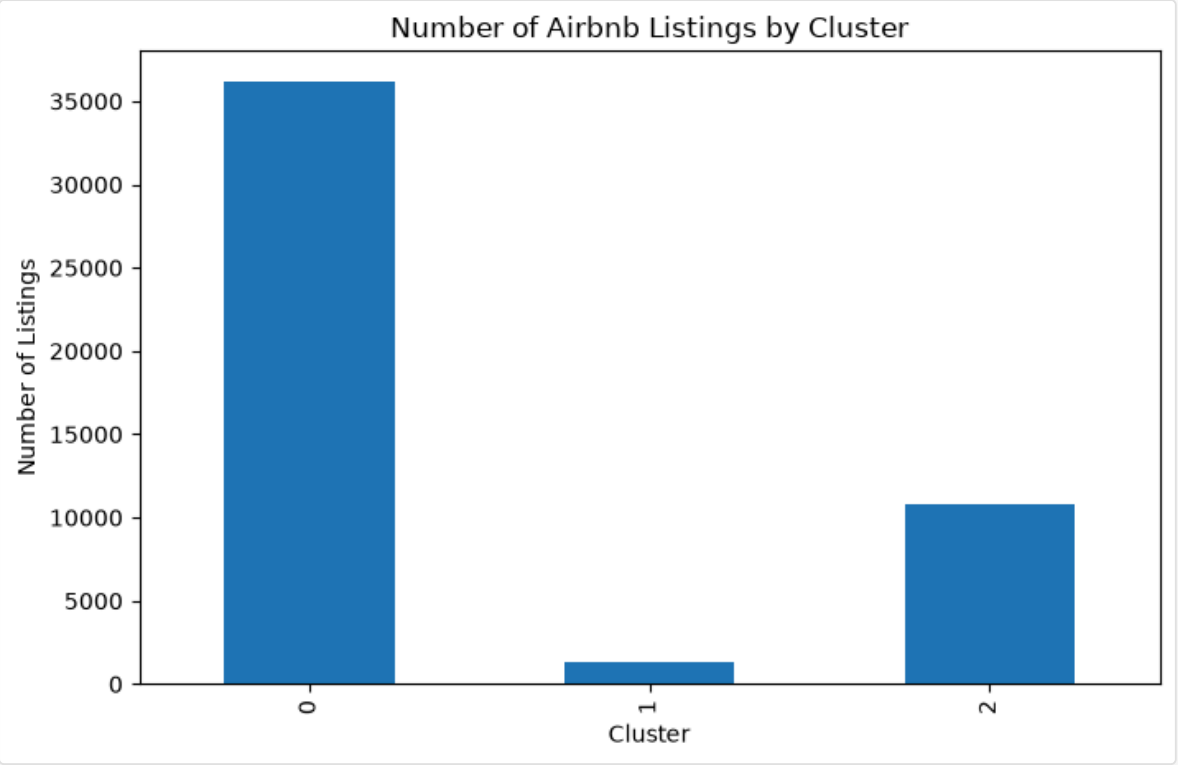


In [51]:

```
cluster_summary["number_of_listings"].plot(kind="bar", figsize=(8, 5))

plt.xlabel("Cluster")
plt.ylabel("Number of Listings")
plt.title("Number of Airbnb Listings by Cluster")
plt.show()
```

Out [51]:



In [52]:

```
# Reduce the scaled clustering features to two principal components for visualization.
# PCA is used only for plotting, not for training the K-Means model.
pca = PCA(n_components=2, random_state=RANDOM_SEED)
X_cluster_pca = pca.fit_transform(X_cluster_scaled)

df_ml["pca_1"] = X_cluster_pca[:, 0]
df_ml["pca_2"] = X_cluster_pca[:, 1]
```

In [53]:

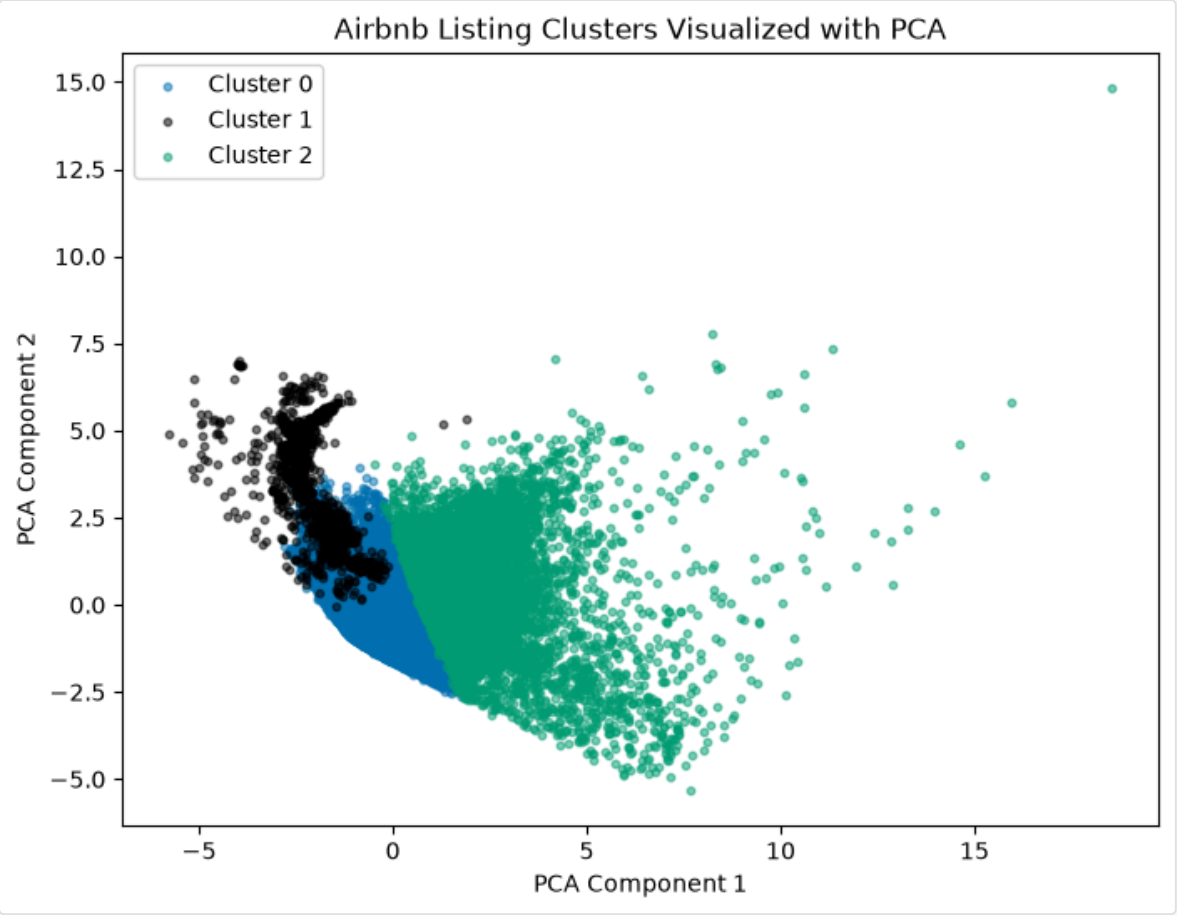
```
# Colour-blind friendly palette for the three clusters.
cluster_colors = {
    0: "#0072B2",  # blue
    1: "#000000",  # black
    2: "#009E73",  # green
}

plt.figure(figsize=(8, 6))

for cluster in sorted(df_ml["cluster"].unique()):
    subset = df_ml[df_ml["cluster"] == cluster]
    plt.scatter(
        subset["pca_1"],
        subset["pca_2"],
        color=cluster_colors[cluster],
        label=f"Cluster {cluster}",
        alpha=0.5,
        s=10,
    )

plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.title("Airbnb Listing Clusters Visualized with PCA")
plt.legend()
plt.show()
```

Out [53]:



In [54]:

```
# Plot clusters geographically to assess whether market segments have spatial patterns.
fig, ax = plt.subplots(figsize=(12, 12))

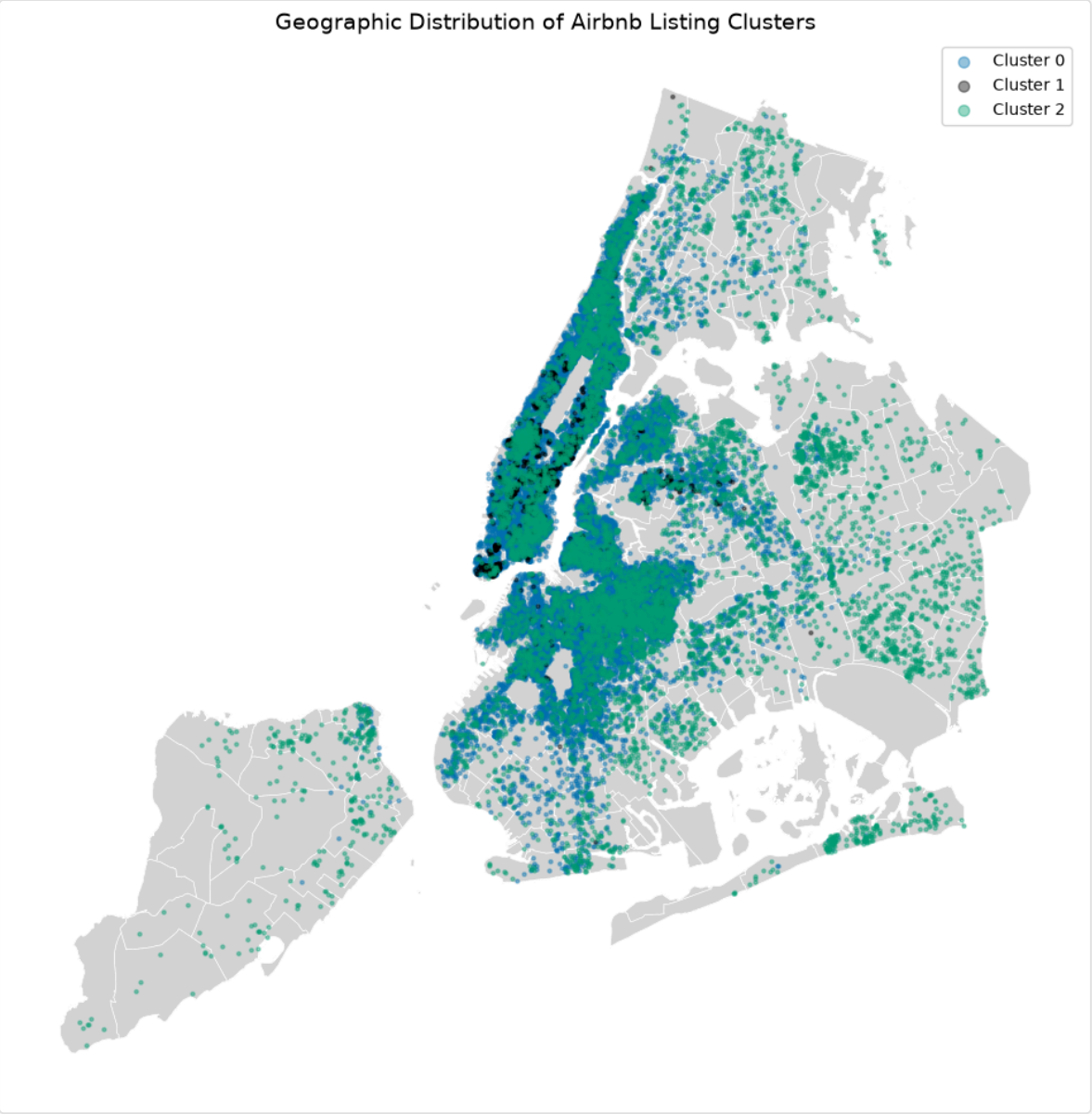
nyc_geojson.plot(ax=ax, color="lightgrey", edgecolor="white", linewidth=0.5)

for cluster in sorted(df_ml["cluster"].unique()):
    subset = df_ml[df_ml["cluster"] == cluster]
    ax.scatter(
        subset["longitude"],
        subset["latitude"],
        color=cluster_colors[cluster],
        s=5,
        alpha=0.4,
        label=f"Cluster {cluster}",
    )

ax.set_title("Geographic Distribution of Airbnb Listing Clusters", fontsize=14)
ax.set_axis_off()
ax.legend(loc="upper right", markerscale=3)

plt.show()
```

Out [54]:



In [55]:

```
features = [
    "latitude",
    "longitude",
    "room_type",
    "boroname",
    "minimum_nights",
    "number_of_reviews",
    "reviews_per_month",
    "calculated_host_listings_count",
    "availability_365",
    "dist_meters_to_subway",
    "dist_min_poi_meters",
]

X_raw = df_ml[features].copy()
y = df_ml["log_price"]

X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X_raw,
    y,
    test_size=0.2,
    random_state=RANDOM_SEED,
)
```

```
categorical_features = ["room_type", "boroname"]

# One-hot encode categorical variables after the train-test split.
X_train = pd.get_dummies(
    X_train_raw,
    columns=categorical_features,
    drop_first=False,
    dtype=int,
)

X_test = pd.get_dummies(
    X_test_raw,
    columns=categorical_features,
    drop_first=False,
    dtype=int,
)

# Ensure the test set has exactly the same columns as the training set.
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)


# Convert predictions back to USD prices.
def rmse_usd(y_true_log, y_pred_log):
    y_true_price = np.expm1(y_true_log)
    y_pred_price = np.expm1(y_pred_log)
    return np.sqrt(mean_squared_error(y_true_price, y_pred_price))


rmse_usd_scorer = make_scorer(
    rmse_usd,
    greater_is_better=False,
)
```

In [56]:

```
model_grids = {
    "Decision Tree": {
        "estimator": DecisionTreeRegressor(random_state=RANDOM_SEED),
        "param_grid": {
            "max_depth": [6, 10, 15],
            "min_samples_split": [20, 50, 100],
            "min_samples_leaf": [10, 20, 50],
        },
    },
    "Random Forest": {
        "estimator": RandomForestRegressor(
            random_state=RANDOM_SEED,
            n_jobs=1,
        ),
        "param_grid": {
            "n_estimators": [100, 300],
            "max_depth": [15, 20, None],
            "min_samples_leaf": [1, 5, 10],
        },
    },
    "Gradient Boosting": {
        "estimator": GradientBoostingRegressor(random_state=RANDOM_SEED),
        "param_grid": {
            "n_estimators": [100, 300],
            "learning_rate": [0.05, 0.1],
            "max_depth": [2, 3, 4],
            "min_samples_leaf": [10, 20],
            "subsample": [0.8, 1.0],
        },
    },
}

grid_summary_results = []
best_estimators = {}
all_grid_results = []

for model_family, config in model_grids.items():
```

```
    print(f"\nRunning GridSearchCV for {model_family}...")

    grid_search = GridSearchCV(
        estimator=config["estimator"],
        param_grid=config["param_grid"],
        scoring=rmse_usd_scorer,
        cv=3,
        n_jobs=-1,
        verbose=0,
        refit=True,
        return_train_score=True,
    )

    grid_search.fit(X_train, y_train)

    best_model_for_family = grid_search.best_estimator_
    best_estimators[model_family] = best_model_for_family

    preds_log = best_model_for_family.predict(X_test)

    y_test_price = np.expm1(y_test)
    preds_price = np.expm1(preds_log)

    mae = mean_absolute_error(y_test_price, preds_price)
    rmse = np.sqrt(mean_squared_error(y_test_price, preds_price))
    r2_usd = r2_score(y_test_price, preds_price)
    r2_log = r2_score(y_test, preds_log)

    grid_summary_results.append(
        {
            "Model Family": model_family,
            "Best CV RMSE_USD": -grid_search.best_score_,
            "Test MAE_USD": mae,
            "Test RMSE_USD": rmse,
            "Test R2_USD": r2_usd,
            "Test R2_Log_Price": r2_log,
            "Best Parameters": grid_search.best_params_,
        }
    )

    family_results = pd.DataFrame(grid_search.cv_results_)
    family_results["Model Family"] = model_family
    family_results["Mean CV RMSE_USD"] = -family_results["mean_test_score"]

    all_grid_results.append(family_results)

model_results = pd.DataFrame(grid_summary_results).sort_values("Test RMSE_USD")

display(model_results)

grid_results_all = pd.concat(all_grid_results, ignore_index=True)

top_grid_results = grid_results_all[
    [
        "Model Family",
        "rank_test_score",
        "Mean CV RMSE_USD",
        "std_test_score",
        "params",
    ]
].sort_values("Mean CV RMSE_USD")

display(top_grid_results.head(10))

best_model_family = model_results.iloc[0]["Model Family"]
best_model = best_estimators[best_model_family]

print(f"Best model based on test RMSE: {best_model_family}")
print("Best parameters:")
print(model_results.iloc[0]["Best Parameters"])

model = best_model
```

```
# Feature importance analysis for the best model:
importances = pd.Series(
    best_model.feature_importances_,
    index=X_train.columns,
).sort_values(ascending=False)

print("\nTop feature importances:")
print(importances.head(15))

importances.head(15).plot(kind="bar", figsize=(10, 5))

plt.xlabel("Feature")
plt.ylabel("Importance")
plt.title(f"Top 15 Feature Importances - {best_model_family}")
plt.xticks(rotation=45, ha="right")
plt.show()

X_all = pd.get_dummies(
    X_raw,
    columns=categorical_features,
    drop_first=False,
    dtype=int,
)

X_all = X_all.reindex(columns=X_train.columns, fill_value=0)

X = X_all

# Export
regression_export = X_raw.copy()
regression_export["log_price"] = y

regression_export.to_csv(
    datasets_path / "airbnb_nyc_regression_features.csv",
    index=False,
)
```

Out [56]:

```
Running GridSearchCV for Decision Tree...

Running GridSearchCV for Random Forest...

Running GridSearchCV for Gradient Boosting...
```

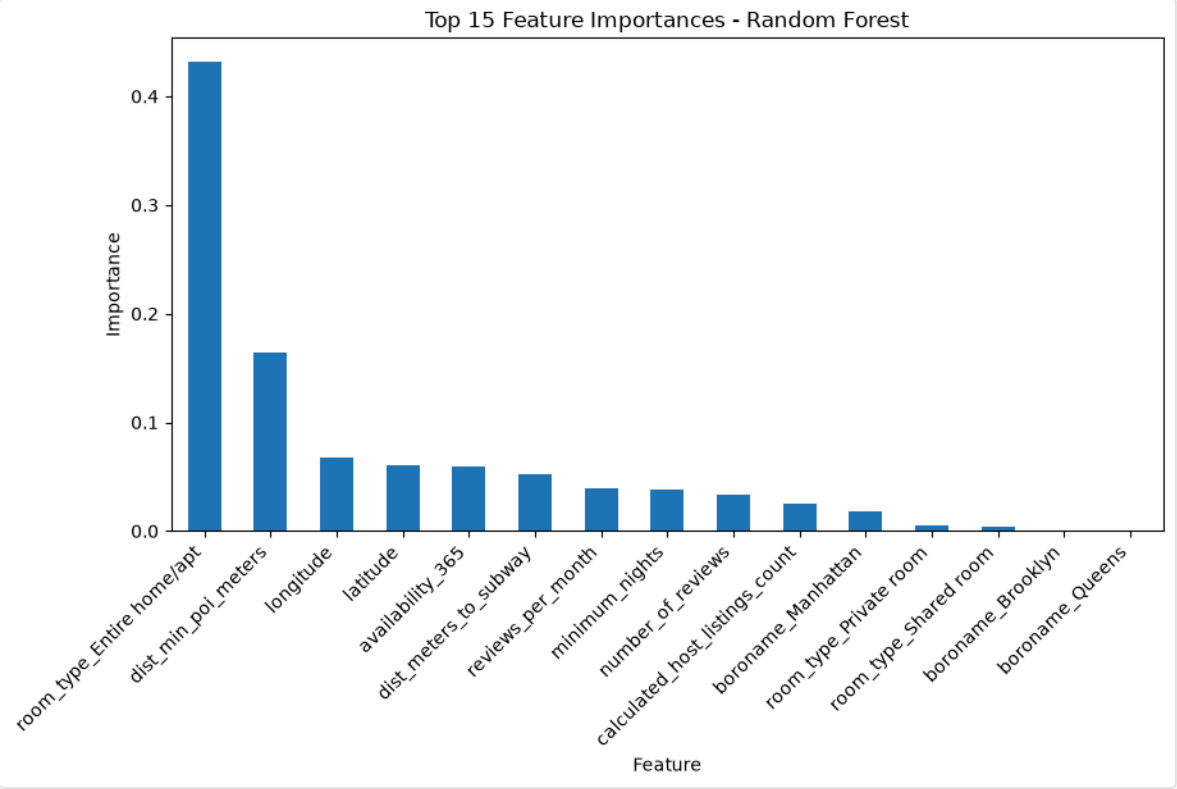
	Model Family	Best CV RMSE_USD	Test MAE_USD	Test RMSE_USD	Test R2_USD	Test R2_Log_Price	Best Parameters
1	Random Forest	74.464572	42.793986	75.523153	0.483954	0.639405	{'max_depth': 20, 'min_samples_le…
2	Gradient Boosting	75.426407	43.358196	77.074301	0.462538	0.638751	{'learning_rate': 0.1, 'max_depth': 4…
0	Decision Tree	78.452225	45.404878	79.008086	0.435230	0.595020	{'max_depth': 10, 'min_samples_le…

	Model Family	rank_test_score	Mean CV RMSE_USD	std_test_score	params
34	Random Forest	1	74.464572	1.052191	{'max_depth': 20, 'min_samples…
40	Random Forest	2	74.520381	1.052844	{'max_depth': None, 'min_sampl…
28	Random Forest	3	74.556285	1.065372	{'max_depth': 15, 'min_samples…
36	Random Forest	4	74.617042	1.174848	{'max_depth': 20, 'min_samples…
42	Random Forest	5	74.618085	1.176060	{'max_depth': None, 'min_sampl…
33	Random Forest	6	74.644150	1.118351	{'max_depth': 20, 'min_samples…
27	Random Forest	7	74.661260	1.157690	{'max_depth': 15, 'min_samples…
30	Random Forest	8	74.668401	1.181884	{'max_depth': 15, 'min_samples…
35	Random Forest	9	74.688812	1.222934	{'max_depth': 20, 'min_samples…
41	Random Forest	10	74.690755	1.224256	{'max_depth': None, 'min_sampl…

```
Best model based on test RMSE: Random Forest
Best parameters:
{'max_depth': 20, 'min_samples_leaf': 1, 'n_estimators': 300}

Top feature importances:
room_type_Entire home/apt         0.432111
dist_min_poi_meters               0.164703
longitude                         0.067062
```

```
latitude                        0.060106
availability_365                0.059109
dist_meters_to_subway           0.052718
reviews_per_month               0.039004
minimum_nights                  0.037737
number_of_reviews               0.033303
calculated_host_listings_count  0.024943
boroname_Manhattan              0.018226
room_type_Private room          0.004582
room_type_Shared room           0.004476
boroname_Brooklyn               0.000770
boroname_Queens                 0.000681
dtype: float64
```



In [57]:

```
best_preds_log = best_model.predict(X_test)
best_preds_price = np.expm1(best_preds_log)
y_test_price = np.expm1(y_test)

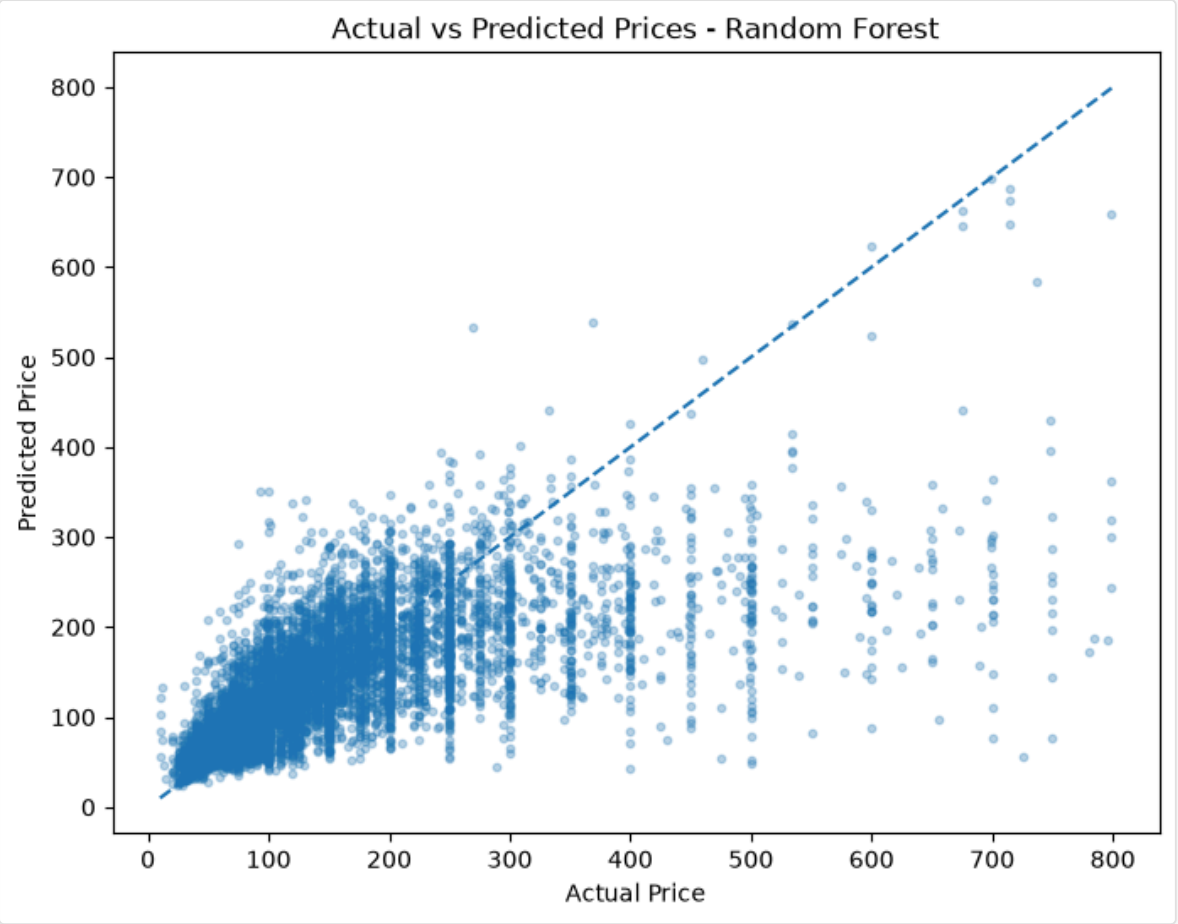
plt.figure(figsize=(8, 6))
plt.scatter(y_test_price, best_preds_price, alpha=0.3, s=10)

min_price = min(y_test_price.min(), best_preds_price.min())
max_price = max(y_test_price.max(), best_preds_price.max())

plt.plot([min_price, max_price], [min_price, max_price], linestyle="--")

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title(f"Actual vs Predicted Prices - {best_model_family}")
plt.show()
```

Out [57]:



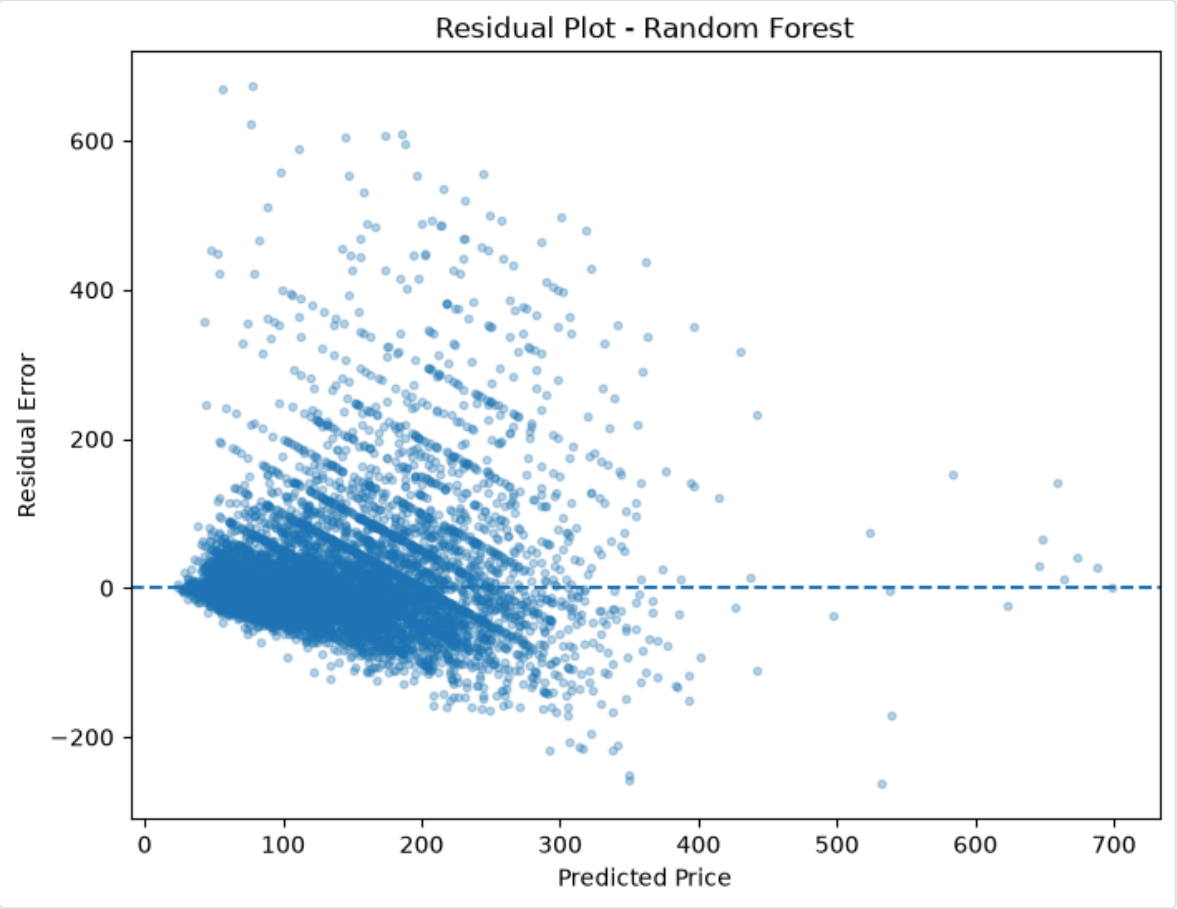
In [58]:

```
residuals = y_test_price - best_preds_price

plt.figure(figsize=(8, 6))
plt.scatter(best_preds_price, residuals, alpha=0.3, s=10)
plt.axhline(0, linestyle="--")

plt.xlabel("Predicted Price")
plt.ylabel("Residual Error")
plt.title(f"Residual Plot - {best_model_family}")
plt.show()
```

Out [58]:



In [59]:

```
# Out-of-fold estimation using the best-performing regression model
oof_log_price = cross_val_predict(
    model, X, y, cv=KFold(n_splits=5, shuffle=True, random_state=RANDOM_SEED)
)
df_ml["predicted_price"] = np.expm1(oof_log_price)
df_ml["residue_error"] = df_ml["price"] - df_ml["predicted_price"]

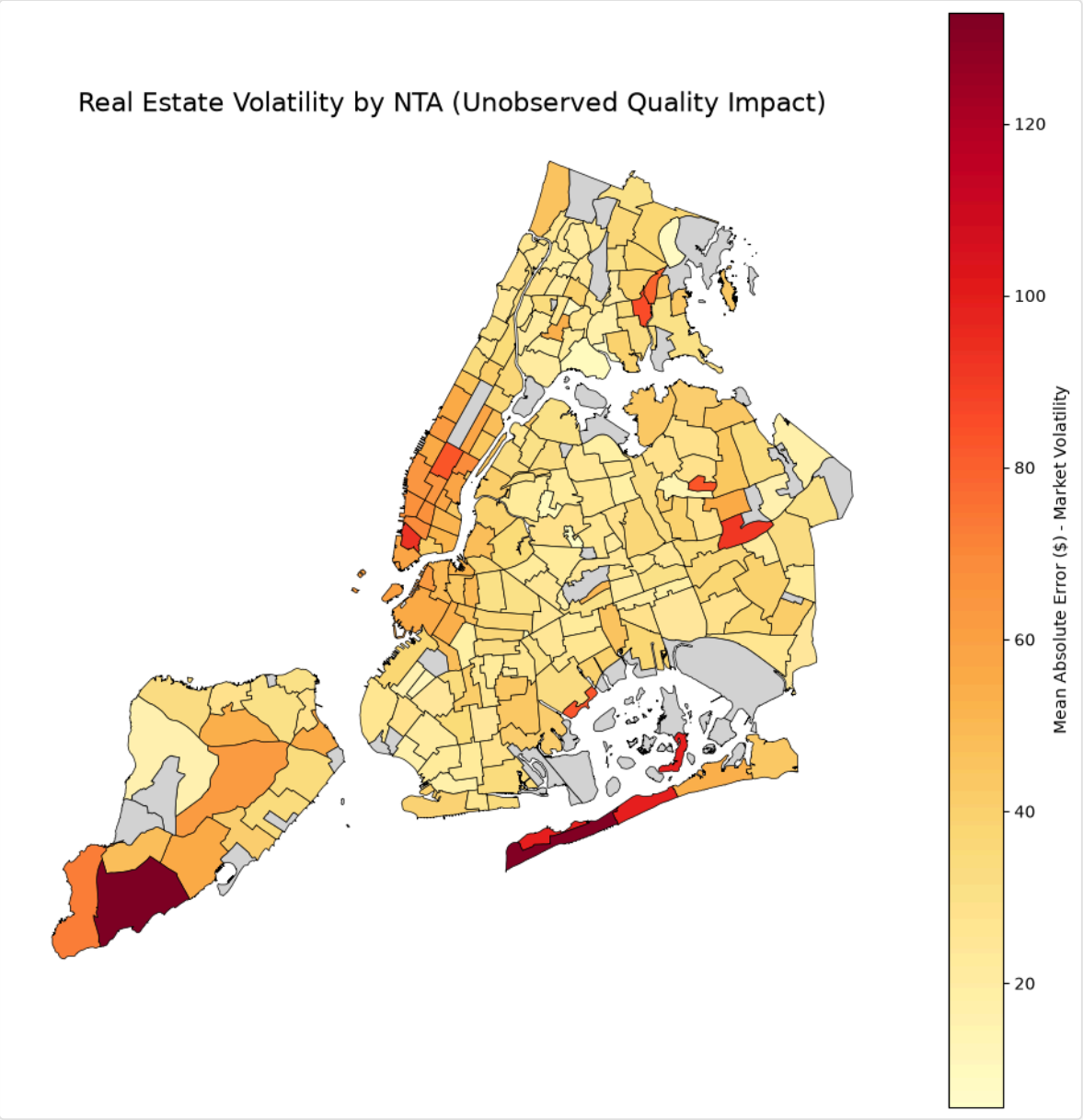
# Maps the mean absolute error per neighborhood to visualize market volatility and unobserved quality impact.
df_ml["absolute_error"] = np.abs(df_ml["residue_error"])
volatility_nta = df_ml.groupby("ntaname")["absolute_error"].mean().reset_index()

volatility_map = nyc_geojson.merge(volatility_nta, on="ntaname", how="left")

fig, ax = plt.subplots(figsize=(12, 12))
volatility_map.plot(
    column="absolute_error",
    cmap="YlOrRd",
    linewidth=0.5,
    edgecolor="black",
    legend=True,
    legend_kwds={"label": "Mean Absolute Error ($) - Market Volatility"},
    missing_kwds={"color": "lightgrey"},
    ax=ax,
)

ax.set_title("Real Estate Volatility by NTA (Unobserved Quality Impact)", fontsize=16)
ax.set_axis_off()
plt.show()
```

Out [59]:



Real Estate Volatility by NTA Explaination

This choropleth map visualises spatial market volatility by aggregating the mean absolute error of the best-performing regression model across neighbourhoods, acting as a risk indicator for the cold-start pricing tool. **High-error zones** highlight areas where unobserved property quality heavily dictates value, **demonstrating that baseline spatial features are insufficient for accurate automated valuation**. By intersecting these geographic error margins with the K-Means clustering results, we can mathematically quantify which specific market segments drive this pricing uncertainty, allowing the platform to confidently deploy automated pricing for highly standardised listings whilst flagging volatile segments that necessitate further qualitative data.